

Chapter 2 – Neural Networks and Static Word Embeddings

Advanced Machine Learning for Finance

Giovanni Della Lunga

University of Bologna

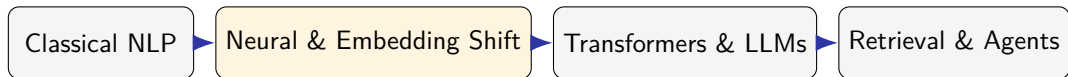
April-May 2026

Introduction

Where We Are in the Course

The general path of the course

We started from **classical text representations**. Today we move to the first major transition: from hand-crafted lexical representations to **learned representations** and to the basic logic of **neural language modeling**.



- ▶ Lesson 1 showed how text becomes data through sparse representations.
- ▶ Lesson 2 asks why sparse representations are not enough and how neural models begin to learn meaning.

Why This Lesson Matters

A conceptual reason

Without a minimum understanding of neural networks, embeddings and LLMs easily look like magic. We need a compact but serious foundation.

A practical reason

Many modern NLP systems are built on top of learned vector representations. Retrieval, ranking, semantic search, and generation all depend on this shift.

Guiding question

How do we move from *counting words* to *learning representations* that encode similarity, context, and structure?

Where We Left Off

In the previous lesson, text was represented mainly through:

- ▶ sparse lexical indicators,
- ▶ term frequencies,
- ▶ inverse document frequencies,
- ▶ geometric similarity in high-dimensional spaces.

What was missing?

These representations are useful, interpretable, and computationally efficient, but they do not **learn** features. The model depends heavily on the representation we engineer by hand.

New idea

What if the system itself could **learn** a useful representation while learning the task?

Learning Objectives of Today's Lesson

By the end of the lesson, students should be able to:

- ▶ explain the difference between a perceptron, logistic regression, and a differentiable neuron;
- ▶ understand why gradient-based learning is a turning point for modern NLP;
- ▶ describe the intuition of distributional semantics and dense word vectors;
- ▶ explain the main logic of Word2Vec and the difference between CBOW and Skip-Gram;
- ▶ understand the role of co-occurrence in GloVe;
- ▶ identify the strengths and limitations of **static** embeddings.

Roadmap of the Lesson

A Short Introduction to Deep Learning

The Elementary Block: Three Instantiations

The same functional form

$$\hat{y} = f(\mathbf{w}^\top \mathbf{x} + b)$$

gives rise to three qualitatively different models depending on the choice of f .

Choice of f	Output	Task
Identity $f(z) = z$	real number	regression
Sign $f(z) = \text{sgn}(z)$	$\{-1, +1\}$	hard classification
Sigmoid $f(z) = \sigma(z)$	$(0, 1)$	probabilistic classification

Reading direction

Each choice removes one assumption of the previous one. The three examples that follow build the conceptual path toward neural networks.

Instantiation 1 — Linear Regression ($f = \text{Id}$)

With $f(z) = z$ the block becomes

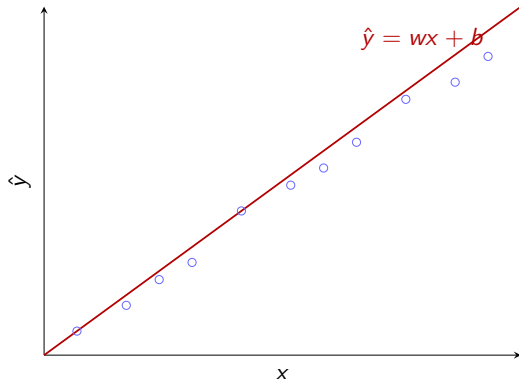
$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b.$$

Setting.

- ▶ Input: a single feature x (e.g. log-market-cap).
- ▶ Output: a continuous target y (e.g. annual return).
- ▶ Loss: mean squared error

$$\mathcal{L}(w, b) = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2.$$

- ▶ Update: gradient descent on $\mathcal{L}$.



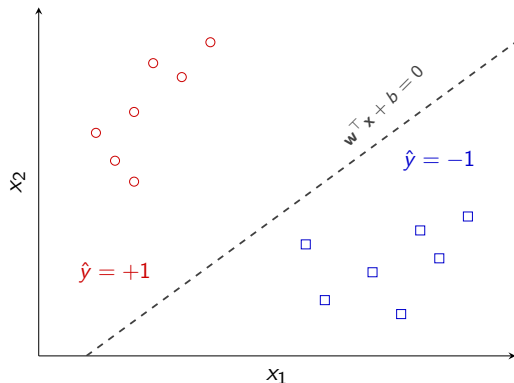
Instantiation 2 — Hard Classifier ($f = \text{sgn}$)

With $f(z) = \text{sgn}(z)$ the block becomes

$$\hat{y} = \text{sgn}(\mathbf{w}^\top \mathbf{x} + b) \in \{-1, +1\}.$$

Setting.

- ▶ Two linearly separable clusters in $\mathbb{R}^2$.
- ▶ The hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ is the decision boundary.
- ▶ Points on opposite sides receive opposite labels.
- ▶ Update rule: correct only on mistakes
 $\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$.



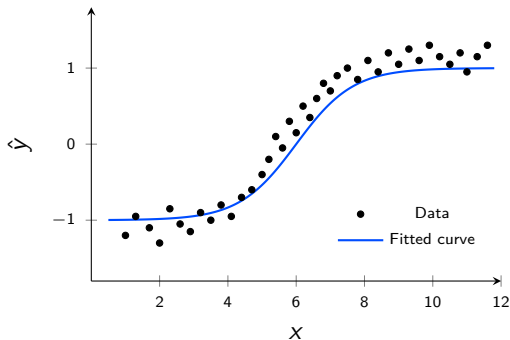
Instantiation 3 — Probabilistic Classifier ($f = \sigma$)

Replacing sgn with the sigmoid $\sigma(z) = 1/(1 + e^{-z})$ gives a **smooth**, differentiable output in $(0, 1)$:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b).$$

- ▶ Same two clusters, but now the model outputs a *probability*, not a hard label.
- ▶ Loss: binary cross-entropy

$$\mathcal{L} = -\frac{1}{m} \sum_i [y_i \log a_i + (1 - y_i) \log(1 - a_i)].$$



Why Neural Models Enter the Picture

The simplest learning systems assign a label directly from the input:

$$\hat{y} = f(\mathbf{w}^\top \mathbf{x} + b).$$

If f is a hard threshold or a linear function, the model draws a **fixed boundary** in the feature space we provide.

The fundamental problem

If the input features do not expose the right structure, no amount of optimization can compensate. The bottleneck is **representation**, not the optimizer.

Why this matters for NLP

Text features engineered by hand — counts, frequencies, indicators — are static. Neural models are important because they can **learn** a better feature space jointly with the task, rather than being handed one.

Three Related but Distinct Objects

To understand how neural networks learn representations, we need to trace a short conceptual path through three objects that are closely related but serve distinct roles.

Object	Main idea
Perceptron	Hard threshold decision rule with mistake-driven updates.
Logistic regression	Probabilistic model for binary classification; smooth, differentiable loss.
Sigmoid neuron	Differentiable building block reusable inside neural networks.

Reading direction

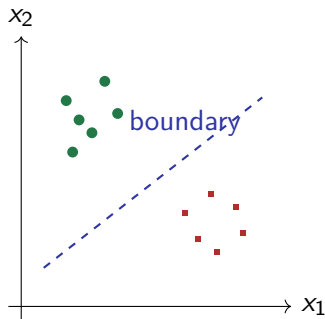
Each object removes one limitation of the previous one.

The Perceptron

- ▶ The perceptron is one of the earliest artificial neuron models.
- ▶ It performs a **hard** decision through a threshold rule:

$$\hat{y} = \mathbb{I}(\mathbf{w}^\top \mathbf{x} + b \geq 0).$$

- ▶ It learns a **linear decision boundary**.
- ▶ It works on linearly separable problems, but fails on XOR-like patterns.



Perceptron Learning Rule

Given a labeled example $(\mathbf{x}_i, y_i)$ with $y_i \in \{0, 1\}$:

$$\hat{y}_i = \mathbb{I}(\mathbf{w}^\top \mathbf{x}_i + b \geq 0)$$

and update only on mistakes:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y_i - \hat{y}_i)\mathbf{x}_i, \quad b \leftarrow b + \eta(y_i - \hat{y}_i).$$

Important clarification

This is **not** gradient descent on cross-entropy. The activation is not differentiable, so the perceptron is historically important but not the right conceptual foundation for modern neural training.

What the Perceptron Is Not

- ▶ It is not a probabilistic model.
- ▶ It is not trained by maximizing likelihood.
- ▶ It is not trained by minimizing cross-entropy with gradient descent.

Then why do we still teach it?

- ▶ It is conceptually simple.
- ▶ It makes the notion of linear separability explicit.
- ▶ It reveals a major limitation: a linear rule cannot solve every pattern.
- ▶ It motivates the move toward differentiable models.

Logistic Regression as a Probabilistic Model

Instead of making a hard decision immediately, logistic regression models the conditional probability of class 1:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(t) = \frac{1}{1 + e^{-t}}.$$

- ▶ The output lies in $(0, 1)$.
- ▶ Labels are interpreted as Bernoulli random variables.
- ▶ The model remains linear in the input, but it is now **smooth and differentiable**.

Where the Loss Comes From

If $a_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$, then:

$$P(y_i \mid \mathbf{x}_i; \mathbf{w}, b) = a_i^{y_i} (1 - a_i)^{1-y_i}.$$

Taking logs and summing over the sample leads to the log-likelihood:

$$\log L(\mathbf{w}, b) = \sum_{i=1}^m [y_i \log a_i + (1 - y_i) \log(1 - a_i)].$$

Maximizing this quantity is equivalent to minimizing the binary cross-entropy:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m [y_i \log a_i + (1 - y_i) \log(1 - a_i)].$$

From Differentiable Activation to Differentiable Loss

The loss for logistic regression is a composition of functions:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y_i \log \underbrace{\sigma(\mathbf{w}^\top \mathbf{x}_i + b)}_{a_i} + (1 - y_i) \log(1 - \underbrace{\sigma(\mathbf{w}^\top \mathbf{x}_i + b)}_{a_i}) \right].$$

Differentiating with respect to w_j via the chain rule:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial \mathcal{L}}{\partial a_i} \cdot \underbrace{\frac{\partial a_i}{\partial z_i}}_{\sigma'(z_i)} \cdot \frac{\partial z_i}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (a_i - y_i) x_{ij}.$$

Why differentiability of σ is essential

The middle term $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ is what makes the chain rule close. If the activation were a hard threshold (as in the perceptron), $\sigma' = 0$ almost everywhere and gradient descent would have no signal to follow.

From Differentiable Activation to Differentiable Loss

Key message

Choosing a **differentiable** activation function is not a cosmetic detail: it is what makes the loss itself differentiable with respect to the model parameters.

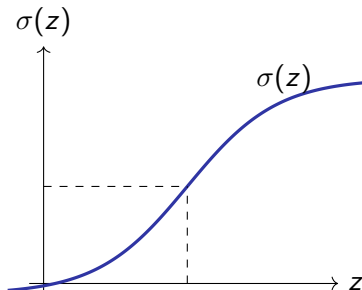
Why Logistic Regression Fits the Neural View

Two crucial ingredients

- ▶ **Differentiability:** $\sigma(\cdot)$ is differentiable.
- ▶ **Smooth objective:** cross-entropy works naturally with gradients.

Bridge to deep learning

Differentiable model + differentiable loss $\Rightarrow$ gradient-based training.



Gradient Descent in Vector Form

For logistic regression, the gradient takes a compact form:

$$\nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{m} X^\top (A - y), \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{m} \mathbf{1}^\top (A - y).$$

- ▶ $A - y$ is the prediction error in probability space.
- ▶ Updating the parameters means moving them against the gradient:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}, \quad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b}.$$

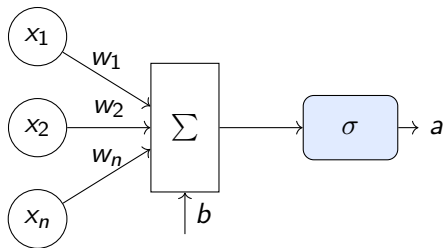
- ▶ The same pattern will later reappear in backpropagation.

The Sigmoid Neuron

A sigmoid neuron performs two operations:

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad a = \sigma(z).$$

- ▶ a linear combination of the inputs,
- ▶ followed by a nonlinear activation.



Why it matters

This is the atomic operation that neural networks will repeat layer after layer.

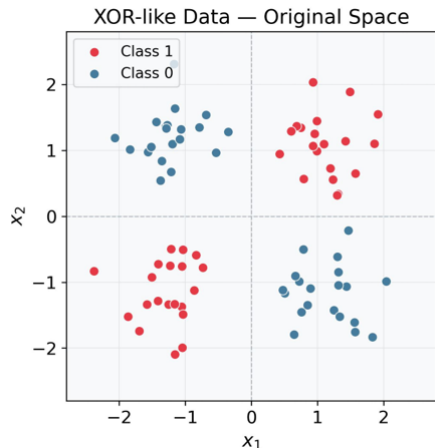
A Limitation of Single-Neuron Models

- ▶ A logistic neuron can optimize its parameters very well.
- ▶ But no amount of optimization can make a single linear boundary solve a truly nonlinear pattern.
- ▶ The limitation is therefore not only optimization – it is also **representation**.

Key idea

If the original coordinates do not expose the right structure, we need a mechanism that learns a **new coordinate system**.

The XOR Problem



Problem definition.

Class assignment depends on a *nonlinear* rule:

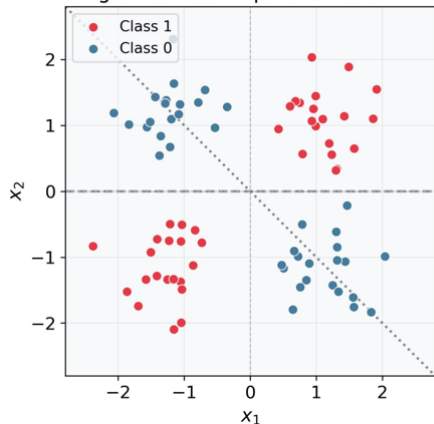
$$y = 1 \text{ iff } x_1 x_2 > 0$$

i.e. points in **Quadrants I and III** belong to **Class 1**, the rest to **Class 0**.

This is the classic **XOR-like** pattern: the label depends on the *product* (interaction) of the two features, not on each feature alone.

Why a Single Neuron Fails

No Single Line Can Separate These Classes



No matter which line we try:

$$w_1x_1 + w_2x_2 + b = 0$$

it **always** misclassifies a substantial fraction of the data.

The problem is not optimization.

A logistic neuron can find the best possible line — but no line is good enough.

The problem is the representation: the raw features (x_1, x_2) do not expose the structure needed for a linear decision.

The Core Idea: Learn a New Coordinate System

Key insight (Goodfellow et al., Ch. 6)

Instead of classifying in the *original* space, transform the input into a **new space** where the problem *becomes* linearly separable.

Introduce a **hidden layer** $h(\mathbf{x})$ with two neurons:

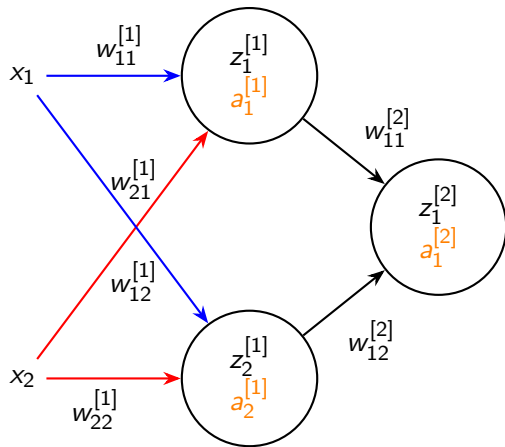
$$h_j(\mathbf{x}) = \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j), \quad j = 1, 2$$

Then classify in the *transformed* space:

$$\mathbf{x} \xrightarrow{h(\mathbf{x})} (h_1, h_2) \xrightarrow{\text{linear}} \hat{y}$$

The hidden layer does not add complexity for its own sake.
It *builds a coordinate system* in which a hyperplane suffices.

The Core Idea: Learn a New Coordinate System

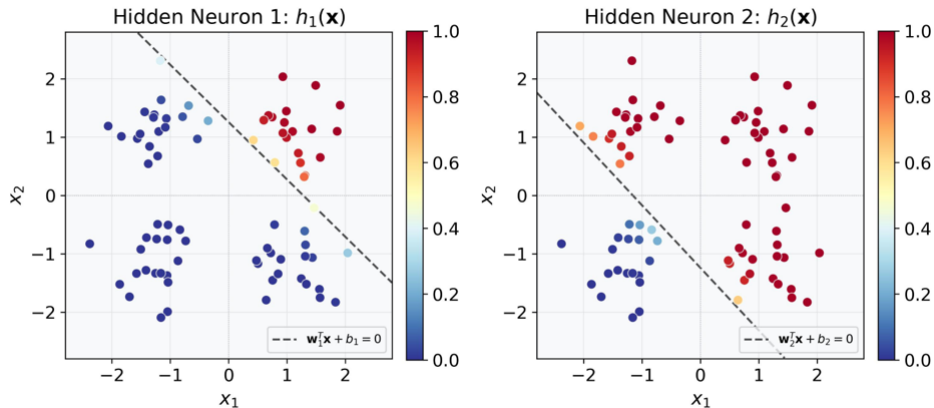


$$z_1^{[1]} = w_{11}^{[1]}x_1 + w_{12}^{[1]}x_2 + b_1^{[1]}$$
$$a_1^{[1]} = \frac{1}{1 + e^{-z_1^{[1]}}}$$

$$z_1^{[2]} = w_{11}^{[2]}a_1^{[1]} + w_{12}^{[2]}a_2^{[1]} + b_1^{[2]}$$
$$a_1^{[2]} = \frac{1}{1 + e^{-z_1^{[2]}}}$$

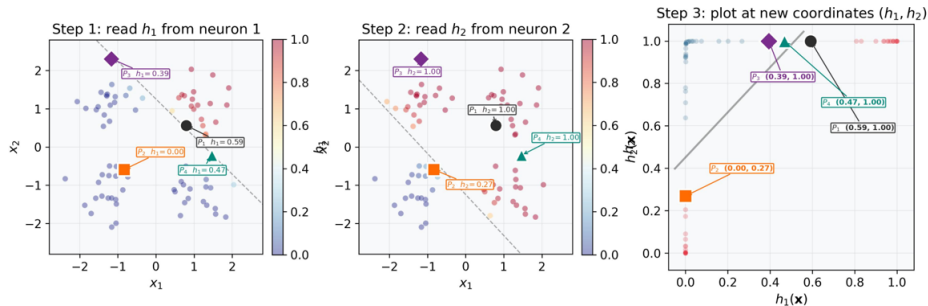
$$z_2^{[1]} = w_{21}^{[1]}x_1 + w_{22}^{[1]}x_2 + b_2^{[1]}$$
$$a_2^{[1]} = \frac{1}{1 + e^{-z_2^{[1]}}}$$

What Each Hidden Neuron Computes



Each neuron partitions the original space with its own *linear boundary* and then “squashes” the result through σ . Individually, neither h_1 nor h_2 separates the classes — but **together** they create a representation where a linear classifier succeeds.

Tracking Points: From (x_1, x_2) to (h_1, h_2)

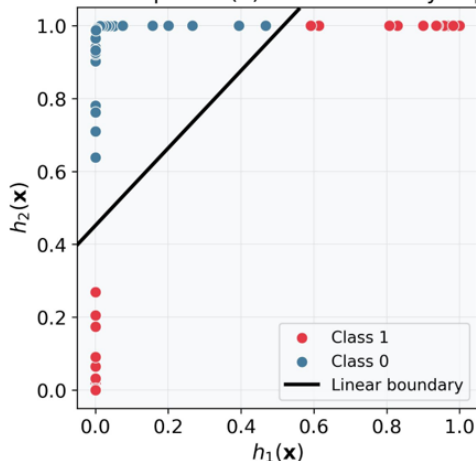


	(x_1, x_2)	Quadrant	h_1	h_2	New coords
P_1	(+0.79, +0.56)	I (cls 1)	0.59	1.00	(0.59, 1.00)
P_2	(-0.83, -0.59)	III (cls 1)	0.00	0.27	(0.00, 0.27)
P_3	(-1.17, +2.31)	II (cls 0)	0.39	1.00	(0.39, 1.00)
P_4	(+1.46, -0.21)	IV (cls 0)	0.47	1.00	(0.47, 1.00)

Recipe: read h_1 from the left panel, h_2 from the center. The pair (h_1, h_2) is the new address in the right panel.

The Transformed Space Is Linearly Separable

Transformed Space $\mathbf{h}(\mathbf{x})$ — Now Linearly Separable!



After the hidden layer, each point $\mathbf{x}$ is mapped to coordinates $(h_1, h_2) \in (0, 1)^2$.

In this new space the two classes occupy **separable clusters**.

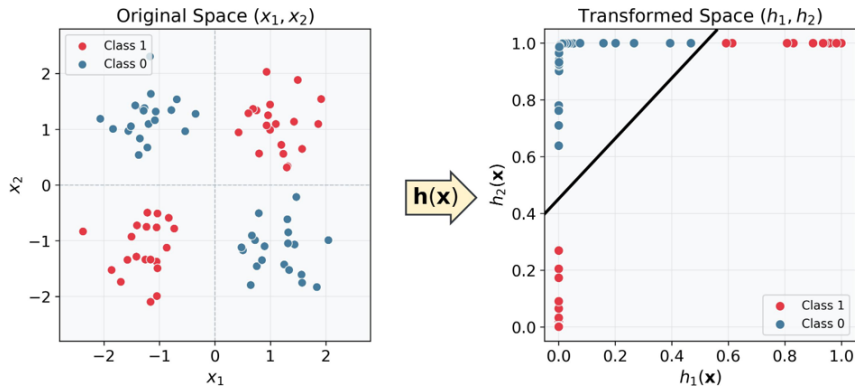
A single linear boundary

$$w'_1 h_1 + w'_2 h_2 + b' = 0$$

now suffices (black line).

This is exactly what the output neuron does: it is a logistic regression in $\mathbf{h}$ -space.

Side by Side: Original $\rightarrow$ Transformed



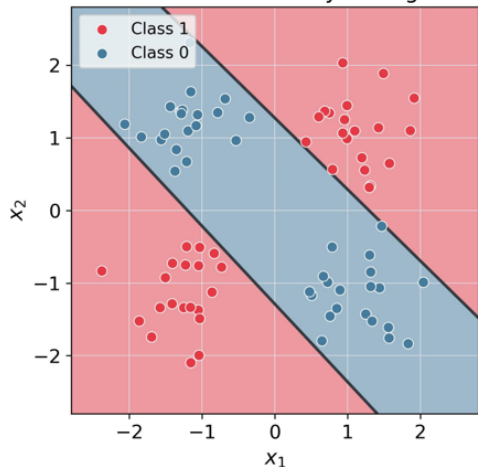
Left: no linear boundary works.

The hidden layer has *learned* the feature interaction that we would have had to engineer by hand (e.g. $x_1 x_2$).

Right: after $h(\mathbf{x})$, a straight line is all we need.

The Decision Boundary Back in the Original Space

Nonlinear Decision Boundary in Original Space



When we map the linear boundary from h -space **back** to (x_1, x_2) -space, it becomes a **nonlinear curve**.

The network has effectively learned:

- 1 A nonlinear feature map $h(\mathbf{x})$
- 2 A linear classifier in that space

Together, these two steps produce a complex decision boundary — while each step alone is simple.

This is the fundamental architecture of *every* feedforward neural network.

From Direct Prediction to Representation Learning

Instead of learning

$$\mathbf{x} \longrightarrow y$$

we can learn

$$\mathbf{x} \longrightarrow h(\mathbf{x}) \longrightarrow y.$$

Interpretation

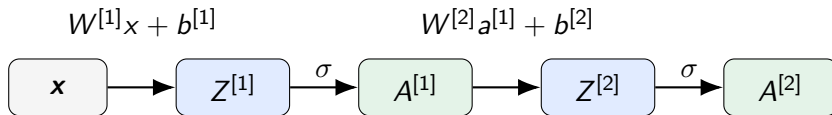
The hidden layer does not add complexity for its own sake. It constructs a new feature space in which a simple linear classifier may become sufficient.

- ▶ Classical machine learning often follows: *feature engineering* $\rightarrow$ *model*.
- ▶ Neural networks aim at: *model* $\rightarrow$ *features* $\rightarrow$ *prediction*.

What a Feedforward Neural Network Really Does

A feedforward network repeatedly applies two steps:

linear map $z = Wx + b \implies$ nonlinear map $a = \phi(z)$.



Core idea

A neural network is a **composition of functions**. Backpropagation is simply the differentiation of this composition.

Remind: Neural Networks Are Supervised Learners

Everything we have built so far is an instance of **supervised learning**.

What supervised learning requires

- ▶ A **labelled dataset** $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$ — inputs paired with known targets.
- ▶ A **model** that maps inputs to predictions: $\hat{y} = f_{\theta}(\mathbf{x})$.
- ▶ A **loss function** $\mathcal{L}(\theta)$ that measures how far $\hat{y}$ is from y .
- ▶ An **optimiser** that adjusts θ to reduce $\mathcal{L}$.

The Optimization Problem and the Training Loop

We have seen that a feedforward network computes

$$\hat{y} = A^{[2]} = \sigma(W^{[2]} \sigma(W^{[1]}X + b^{[1]}) + b^{[2]}).$$

The binary cross-entropy loss measures how far the predictions are from the labels:

$$\mathcal{L}(W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}) = -\frac{1}{m} \sum_{i=1}^m [y_i \log A_i^{[2]} + (1 - y_i) \log(1 - A_i^{[2]})].$$

What we need

To apply gradient descent we must compute

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}}, \quad \frac{\partial \mathcal{L}}{\partial b^{[1]}}, \quad \frac{\partial \mathcal{L}}{\partial W^{[2]}}, \quad \frac{\partial \mathcal{L}}{\partial b^{[2]}}.$$

The Optimization Problem and the Training Loop

The abstract supervised learning cycle instantiates, for our two-layer network, as follows.

One training step

- 1 **Forward pass.** Given a batch $\{(\mathbf{x}^{(i)}, y^{(i)})\}$, compute:

$$A^{[1]} = \sigma(W^{[1]}X + b^{[1]}), \quad A^{[2]} = \sigma(W^{[2]}A^{[1]} + b^{[2]}).$$

- 2 **Loss.** Compute the binary cross-entropy $\mathcal{L}(A^{[2]}, y)$.
- 3 **Backward pass.** Apply backpropagation to obtain $\nabla_{W^{[l]}} \mathcal{L}$, $\nabla_{b^{[l]}} \mathcal{L}$.
- 4 **Update.** $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$.

What this loop optimises

At every step the network is shown **true labels**. The loss measures how wrong the current predictions are.

Backpropagation distributes the blame back to each parameter. The update corrects the parameters in the direction that reduces the error.

After many steps the network learns both *how to predict* and *how to represent the input*.

The Optimization Problem and the Training Loop

Compact summary

labels $\rightarrow$ **loss** $\rightarrow$ **gradient** $\rightarrow$ **update** (repeated for many batches and epochs)

The challenge

$\mathcal{L}$ depends on $W^{[1]}$ only through a long chain of compositions. Backpropagation is the systematic application of the chain rule along that chain.

The Computational Graph Behind Backpropagation

All derivatives we are about to compute follow the same path:

$$W^{[1]} \rightarrow Z^{[1]} \rightarrow A^{[1]} \rightarrow Z^{[2]} \rightarrow A^{[2]} \rightarrow L$$

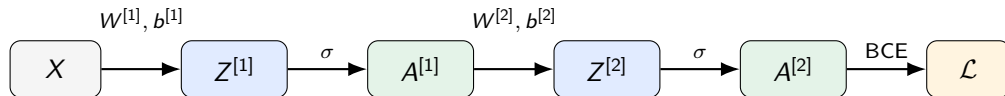
Backpropagation simply applies the chain rule along this graph:

$$\frac{\partial L}{\partial W^{[1]}} = \frac{\partial L}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial A^{[1]}} \frac{\partial A^{[1]}}{\partial Z^{[1]}} \frac{\partial Z^{[1]}}{\partial W^{[1]}}$$

We are not computing many formulas. We are repeating the same rule along a path.

The Two-Layer Network We Will Differentiate

The derivations on the next two slides refer to this specific architecture.



The complete forward pass in matrix form:

$$Z^{[1]} = W^{[1]}X + b^{[1]},$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]},$$

$$\mathcal{L} = -\frac{1}{m} \sum [y \log A^{[2]} + (1 - y) \log(1 - A^{[2]})].$$

$$A^{[1]} = \sigma(Z^{[1]}),$$

$$A^{[2]} = \sigma(Z^{[2]}),$$

What backpropagation computes

Starting from $\mathcal{L}$ and moving **right to left** along the graph, it applies the chain rule at each node to obtain $\partial\mathcal{L}/\partial W^{[2]}$, $\partial\mathcal{L}/\partial b^{[2]}$, $\partial\mathcal{L}/\partial W^{[1]}$, $\partial\mathcal{L}/\partial b^{[1]}$.

Applying the Chain Rule: Layer 2

Referring to the forward-pass graph on the previous slide, for the **output layer** the chain rule gives:

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \underbrace{\frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}}}_{dZ2} \cdot \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = dZ2 \cdot \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$

Applying the Chain Rule: Layer 1

For the **hidden layer**, the chain extends further back through the graph:

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \underbrace{dZ^{[2]} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}}}_{dZ^{[1]}} \cdot \frac{\partial Z^{[1]}}{\partial W^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = dZ^{[1]} \cdot \frac{\partial Z^{[1]}}{\partial b^{[1]}}$$

Observe the recursive pattern:

- ▶ Each layer computes a *local error signal* $dZ^{[\ell]}$ from the layer above
- ▶ The gradients for weights and biases follow the same form at every layer
- ▶ This is the algorithm: **propagate dZ backward, compute gradients locally**

Backpropagation Through Two Layers

Complete Gradient Descent Update — Putting It All Together

Step 1: Compute Gradients (backward pass)

Output Layer ($\ell = 2$):

$$dZ^{[2]} = A^{[2]} - y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} \cdot A^{[1]\top}$$

$$db^{[2]} = \frac{1}{m} \sum dZ^{[2]}$$

Hidden Layer ($\ell = 1$):

$$dZ^{[1]} = W^{[2]\top} \cdot dZ^{[2]} \odot A^{[1]}(1 - A^{[1]})$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} \cdot X^\top$$

$$db^{[1]} = \frac{1}{m} \sum dZ^{[1]}$$

Step 2: Update Parameters

Output Layer ($\ell = 2$):

$$W^{[2]} \leftarrow W^{[2]} - \eta \cdot dW^{[2]}$$

$$b^{[2]} \leftarrow b^{[2]} - \eta \cdot db^{[2]}$$

Hidden Layer ($\ell = 1$):

$$W^{[1]} \leftarrow W^{[1]} - \eta \cdot dW^{[1]}$$

$$b^{[1]} \leftarrow b^{[1]} - \eta \cdot db^{[1]}$$

η = learning rate (hyperparameter)

Repeat until convergence or early stopping.

Backpropagation Through Two Layers

Explicit Update Equations — Fully Expanded Form

Substituting the gradient expressions into $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$:

Output Layer ($\ell = 2$):

$$\mathbf{W}^{[2]} \leftarrow \mathbf{W}^{[2]} - \frac{\eta}{m} \cdot (\mathbf{A}^{[2]} - y) \cdot \mathbf{A}^{[1]\top} \quad (1)$$

$$\mathbf{b}^{[2]} \leftarrow \mathbf{b}^{[2]} - \frac{\eta}{m} \cdot \sum (\mathbf{A}^{[2]} - y) \quad (2)$$

Hidden Layer ($\ell = 1$):

$$\mathbf{W}^{[1]} \leftarrow \mathbf{W}^{[1]} - \frac{\eta}{m} \cdot \underbrace{\left[\mathbf{W}^{[2]\top} \cdot (\mathbf{A}^{[2]} - y) \right]}_{\text{error backpropagated}} \odot \underbrace{\mathbf{A}^{[1]}(1 - \mathbf{A}^{[1]})}_{\text{local } \sigma'} \cdot \mathbf{X}^{\top} \quad (3)$$

$$\mathbf{b}^{[1]} \leftarrow \mathbf{b}^{[1]} - \frac{\eta}{m} \cdot \sum \left[\mathbf{W}^{[2]\top} \cdot (\mathbf{A}^{[2]} - y) \odot \mathbf{A}^{[1]}(1 - \mathbf{A}^{[1]}) \right] \quad (4)$$

Why the Sigmoid Is Not Enough

We have used the sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$ throughout.

Problems in deep networks:

- ▶ **Vanishing gradients:** $\sigma'(z) \leq 0.25$ always. In a network with L layers, gradients shrink as $\sim (0.25)^L$.
- ▶ **Not zero-centered:** outputs are in $(0, 1)$, causing zig-zag updates.
- ▶ **Saturation:** for large $|z|$, the gradient is nearly zero $\Rightarrow$ neurons stop learning.

Key insight

Modern deep learning replaces the sigmoid with activation functions that do **not** suffer from these issues.

The ReLU Activation

The **Rectified Linear Unit** (Nair & Hinton, 2010):

$$g(z) = \max(z, 0)$$

Advantages:

- ▶ Gradient is 1 for $z > 0 \Rightarrow$ no vanishing gradient
- ▶ Very fast to compute
- ▶ Induces **sparsity**: negative pre-activations are set to zero
- ▶ Empirically the most widely used activation

Derivative:

$$g'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$$

Limitation: “dead neurons” — if $z < 0$ for all training inputs, the gradient is permanently zero.

ReLU Variants

Leaky ReLU (Maas et al., 2013)

$$g(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases} \quad \text{with } \alpha \approx 0.01$$

Prevents dead neurons by allowing a small gradient when $z < 0$.

ELU — Exponential Linear Unit (Clevert et al., 2015)

$$g(z) = \begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \leq 0 \end{cases}$$

Smooth approximation that pushes mean activations closer to zero.

Swish / SiLU (Ramachandran et al., 2017)

$$g(z) = z \cdot \sigma(z)$$

Smooth, non-monotonic. Often outperforms ReLU in very deep networks.

Choosing an Activation Function — Practical Guidelines

Activation	Range	Vanishing?	Typical use
Sigmoid $\sigma(z)$	$(0, 1)$	Yes	Output layer (binary)
Tanh	$(-1, 1)$	Yes	Recurrent networks
ReLU	$[0, \infty)$	No	Hidden layers (default)
Leaky ReLU	$(-\infty, \infty)$	No	Hidden layers
Softmax	$(0, 1)^K$	—	Output (multiclass)
Linear	$(-\infty, \infty)$	No	Output (regression)

Rule of thumb:

- ▶ **Hidden layers:** start with ReLU
- ▶ **Output layer:** match the problem (sigmoid for binary, softmax for multiclass, linear for regression)

Distributional Semantics and Dense Representations

Why This Matters for NLP

- ▶ Neural networks are not introduced here to classify toy points forever.
- ▶ They matter because they allow us to **learn dense representations** of symbols, words, contexts, and documents.
- ▶ This is precisely the step that leads from classical vectorization to embeddings.

Transition

Once we accept the idea that a model can learn an internal representation, the next question becomes: *what kind of representation should language units have?*

From Sparse Indicators to Dense Vectors

Sparse representation

- ▶ one-hot vectors,
- ▶ bag-of-words counts,
- ▶ TF-IDF features.

These are high-dimensional and mostly zero.

Dense representation

- ▶ few dimensions,
- ▶ continuous values,
- ▶ proximity meant to reflect similarity.

These are compact and learnable.

Shift in perspective

In sparse models, features are mostly specified by the analyst. In dense models, the representation itself is often learned from data.

The Distributional Hypothesis

Core intuition

Words that occur in similar contexts tend to have similar meanings.

- ▶ This is the conceptual basis of **distributional semantics**.
- ▶ Meaning is not assumed to be given by a dictionary definition alone.
- ▶ Instead, meaning is inferred from patterns of co-occurrence and context.

Example

If the words *bond*, *yield*, *coupon*, and *maturity* frequently appear in related environments, a representation learner should place them near one another in a semantic space.

What Is an Embedding?

- ▶ An embedding maps a symbolic object into a vector in a continuous space.
- ▶ In NLP, the object may be a word, subword, sentence, document, or even an entire query.
- ▶ The main goal is to encode **similarity**, **structure**, and sometimes **task-relevant information** into geometry.

For words

A word embedding associates each token w with a vector

$$\mathbf{e}(w) \in \mathbb{R}^d,$$

where d is much smaller than the vocabulary size.

Why Embeddings Are Attractive

Compared with one-hot vectors

- ▶ lower dimensionality,
- ▶ continuous geometry,
- ▶ similarity can be measured naturally,
- ▶ information can be shared across words.

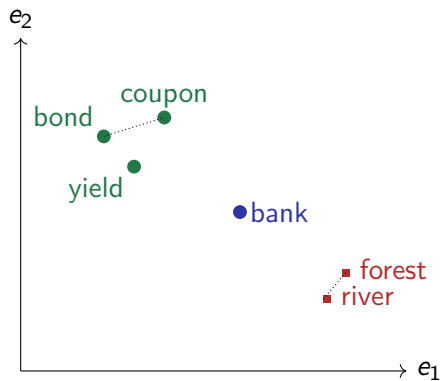
Compared with count-based features

- ▶ better handling of lexical variation,
- ▶ more flexible use in neural models,
- ▶ easier composition into larger architectures.

But note

An embedding is not meaning itself. It is a learned **representation** that may capture useful regularities of meaning.

A Geometric View of Meaning



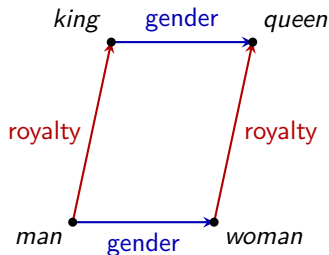
Interpretation : If training succeeds, words used in related contexts should occupy nearby regions of the vector space, while unrelated words should be farther apart.

Vector Arithmetic in Embedding Space

One of the most discussed properties of Word2Vec embeddings is the emergence of **linear relational structure**.

The classic example:

$$\mathbf{e}(\text{king}) - \mathbf{e}(\text{man}) + \mathbf{e}(\text{woman}) \approx \mathbf{e}(\text{queen}).$$



Cosine Similarity Revisited

If two embeddings are represented by vectors $\mathbf{u}$ and $\mathbf{v}$, a common similarity score is

$$\cos(\theta) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}.$$

- ▶ values close to 1 indicate strong alignment,
- ▶ values close to 0 indicate weak relation,
- ▶ values below 0 indicate opposition in direction.

Why cosine?

It focuses on orientation rather than raw magnitude, which is often appropriate when we interpret vectors as semantic directions.

Static Word Embeddings

The first widely successful generation of dense word embeddings was **static**:

- ▶ one word,
- ▶ one vector,
- ▶ regardless of the sentence in which the word appears.

Examples

- ▶ Word2Vec,
- ▶ GloVe,
- ▶ other pre-trained embedding collections.

Historical importance

These models marked the transition from manually engineered lexical spaces to neural representation learning.

Word2Vec: Learning Embeddings with a Shallow Neural Network

What Is Word2Vec?

Word2Vec is a method for learning word embeddings through a **shallow neural architecture**. It transforms a symbolic word into a dense vector:

$$w \mapsto e(w) \in \mathbb{R}^k, \quad k \ll |V|,$$

where $|V|$ is the vocabulary size.

Why it was influential

It made neural embedding learning computationally efficient and practically useful at scale — and it showed that **meaningful geometry can emerge from a simple predictive task**.

Key question

Where do the numbers in $e(w)$ come from, and why can a row of a weight matrix be interpreted as the representation of a word?

The Training Principle: Context as a Proxy for Meaning

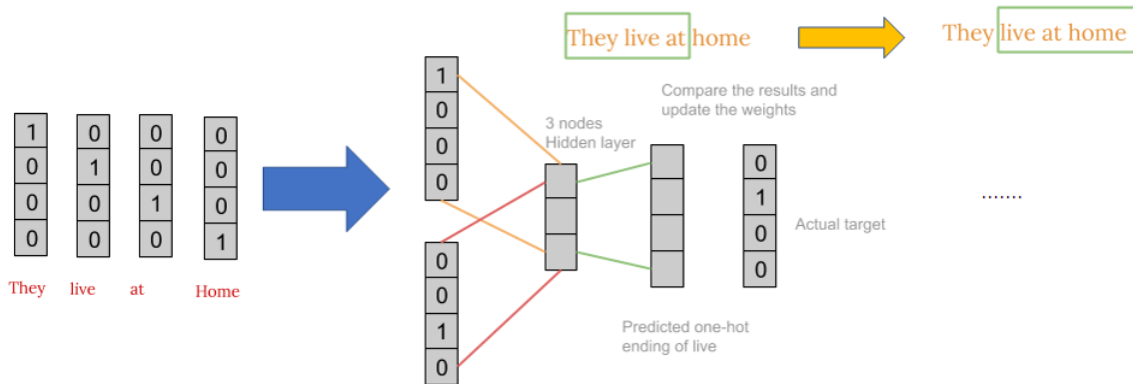
- ▶ The input of word2vec is a text corpus and its output is a set of vectors known as feature vectors that represent words in that corpus.
- ▶ The Word2Vec objective function causes the words that have a similar context to have similar embeddings. Thus in this vector space, these words are really close.
- ▶ Word2vec is not a single algorithm but a combination of two techniques - CBOW(Continuous bag of words) and Skip-gram model.
- ▶ Both these techniques are shallow neural networks that learn weights which act as word vector representations.

The Training Principle: Context as a Proxy for Meaning

- ▶ CBOW predicts the probability of a word to occur **given the words surrounding it**. We can consider a single word or a group of words. But for simplicity, we will take a single context word and try to predict a single target word.
- ▶ The English language contains almost 1.2 million words, making it impossible to include so many words in our example. So I will consider a small example in which we have only four words i.e. **live**, **home**, **they** and **at**. For simplicity, we will consider that the corpus contains only one sentence, that being, **They live at home**.

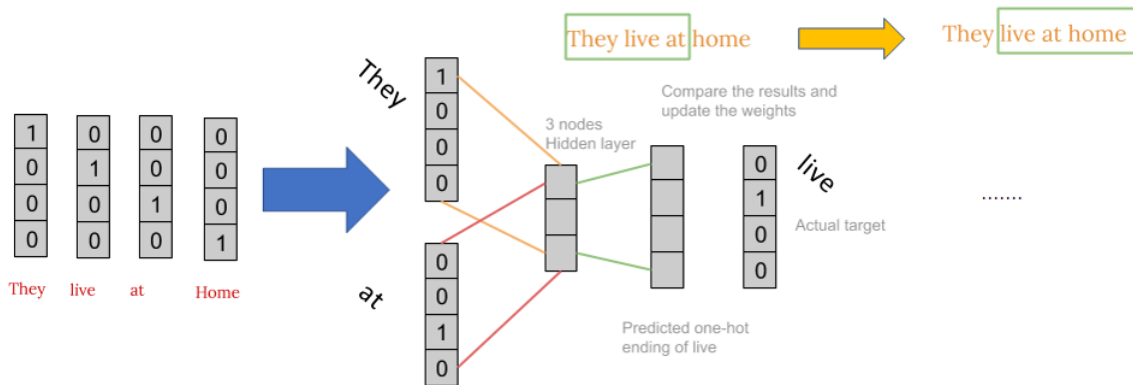
Word2Vec : CBOW approach

First, we convert each word into a one-hot encoding form. Also, we will not consider all the words in the sentence but we only take certain words that are in a window.



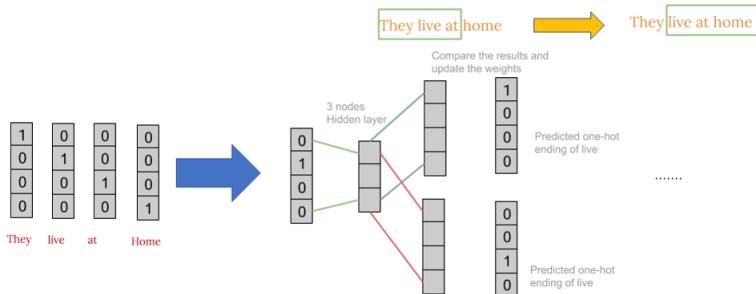
Word2Vec : CBOW approach

For example, for a window size equal to three, we only consider three words in a sentence. The middle word is to be predicted and the surrounding two words are fed into the neural network as context. The window is then slide and the process is repeated again.



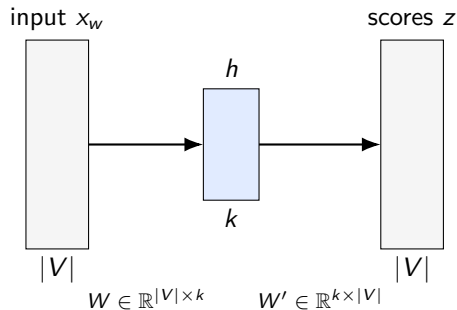
Word2Vec : Skip-Gram Model

- ▶ The Skip-gram model tries to predict the source context words (surrounding words) given a target word (the centre word)
- ▶ The working is conceptually similar to the CBOW, there is just a difference in the architecture of its NN and in the way the weight matrix is generated:



The Architecture: Two Weight Matrices

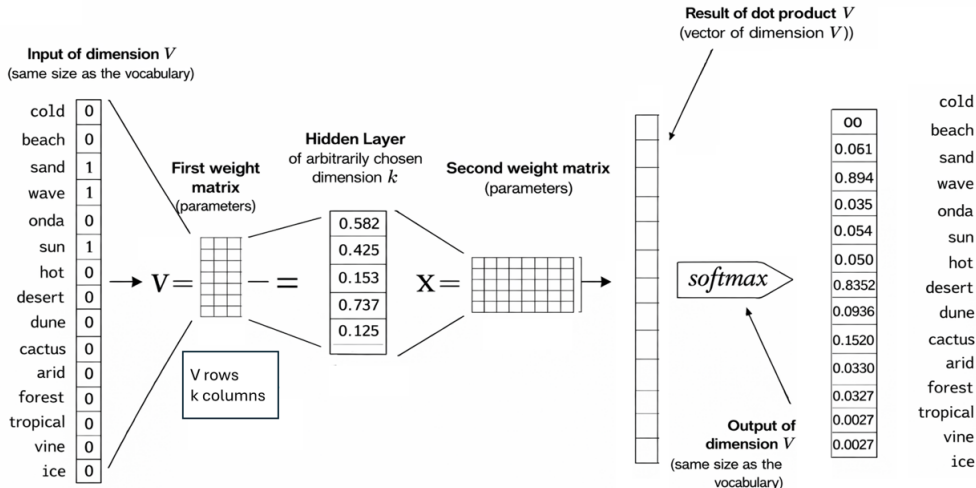
Both model have a minimal structure: $x_w \xrightarrow{W} h \xrightarrow{W'} z$



The key structural fact

$k \ll |V|$. The hidden layer is a **bottleneck**: it forces each word into a compact k -dimensional representation. The rows of W are the embeddings.

The Architecture: Two Weight Matrices



Step 1: from a one-hot vector to the hidden layer

The input word w is represented by a one-hot vector:

$$\mathbf{x}_w = (0, \dots, 0, 1, 0, \dots, 0)^\top \in \mathbb{R}^V.$$

The hidden layer is computed as:

$$\mathbf{h} = W^\top \mathbf{x}_w.$$

Because $\mathbf{x}_w$ has a single entry equal to 1, this multiplication simply selects the row of W corresponding to word w .

Therefore:

$$\mathbf{h} = \mathbf{v}_w.$$

Interpretation. The hidden layer does not create a new arbitrary representation. It simply retrieves the embedding associated with the input word from the matrix W .

What does the hidden layer really do?

Let the i -th row of W be the vector associated with the i -th word in the vocabulary:

$$W = \begin{pmatrix} - & - & - & \mathbf{v}_1^\top & - & - & - \\ - & - & - & \mathbf{v}_2^\top & - & - & - \\ & & & \vdots & & & \\ - & - & - & \mathbf{v}_V^\top & - & - & - \end{pmatrix}.$$

If word w is the j -th word in the vocabulary, then $\mathbf{x}_w = \mathbf{e}_j$, where $\mathbf{e}_j$ is the j -th canonical basis vector. Hence:

$$\mathbf{h} = W^\top \mathbf{e}_j = \mathbf{v}_j.$$

So the hidden representation is nothing more than a **lookup operation** implemented as matrix multiplication. This is why the embedding matrix can be interpreted as a table storing one vector for each word.

Step 2: computing the output scores

Once the hidden representation $\mathbf{h}$ has been obtained, the model computes a score for every candidate context word. The vector of pre-softmax scores is:

$$\mathbf{u} = W'^{\top} \mathbf{h} \in \mathbb{R}^V.$$

The score associated with a specific candidate context word c is the c -th component of $\mathbf{u}$:

$$u_c = \mathbf{v}'_c{}^{\top} \mathbf{h}.$$

Since we already know that $\mathbf{h} = \mathbf{v}_w$, we obtain:

$$u_c = \mathbf{v}'_c{}^{\top} \mathbf{v}_w.$$

By definition the score is the **dot product** between:

- ▶ the input embedding of the central word w ,
- ▶ the output embedding of the candidate context word c .

Recall the identity:

$$\mathbf{v}'_c{}^\top \mathbf{v}_w = \|\mathbf{v}'_c\| \|\mathbf{v}_w\| \cos \theta,$$

where:

- ▶ $\|\mathbf{v}'_c\|$ is the norm of the output embedding,
- ▶ $\|\mathbf{v}_w\|$ is the norm of the input embedding,
- ▶ θ is the angle between the two vectors.

Therefore the score is large when:

- 1 the two vectors point in similar directions, that is, $\cos \theta$ is large,
- 2 and/or the vectors have large norms.

Interpretation. Word2vec learns a latent space in which word-context pairs that frequently co-occur tend to have vectors with a large dot product.

Step 3: from scores to probabilities

The scores are transformed into conditional probabilities through the softmax:

$$P(c \mid w) = \frac{\exp(u_c)}{\sum_{j=1}^V \exp(u_j)}.$$

Substituting the expression for each score gives:

$$P(c \mid w) = \frac{\exp(\mathbf{v}'_c{}^\top \mathbf{v}_w)}{\sum_{j=1}^V \exp(\mathbf{v}'_j{}^\top \mathbf{v}_w)}.$$

Hence the probability assigned to a context word depends directly on the dot product between the two embeddings.

Important consequence. Increasing the dot product between $\mathbf{v}_w$ and $\mathbf{v}'_c$ increases the probability that the model assigns to observing c in the context of w .

Why this interpretation is conceptually useful

The dot product acts as a **compatibility score**.

If two words are statistically compatible in the training corpus, the optimization process pushes their corresponding vectors so that:

$$\mathbf{v}_c^T \mathbf{v}_w$$

becomes large.

This means that word2vec does not directly encode symbolic linguistic rules. Instead, it learns distributed vector representations in which:

- ▶ similar contexts produce similar embeddings,
- ▶ useful semantic or syntactic regularities emerge from the geometry of the learned space,
- ▶ the score is a simple algebraic measure of how well one word predicts another.

A subtle but important point: two embedding spaces

A common simplification is to speak about **the** embedding of a word. During training, however, word2vec maintains two different embeddings for each word:

Representation	Symbol	Role
Input embedding	$\mathbf{v}_w$	used when the word is the center word
Output embedding	$\mathbf{v}'_w$	used when the word is a context word

So the score is more precisely:

$$\text{score}(w, c) = \mathbf{v}'_c{}^\top \mathbf{v}_w.$$

After training, one usually keeps:

- ▶ only the input embeddings,
- ▶ or only the output embeddings,
- ▶ or some combination of both.

Matrix view of the whole mechanism

The entire forward pass of the skip-gram model can be summarized compactly as:

$$\mathbf{x}_w \in \mathbb{R}^V \longrightarrow \mathbf{h} = W^\top \mathbf{x}_w \in \mathbb{R}^k \longrightarrow \mathbf{u} = W'^\top \mathbf{h} \in \mathbb{R}^V.$$

For a specific context word c :

$$u_c = \mathbf{v}'_c{}^\top \mathbf{v}_w.$$

Therefore the skip-gram architecture can be read as:

select the embedding of the input word $\rightarrow$ compare it with all output embeddings by dot products.

This is the fundamental reason why the final score has a natural interpretation as a dot product between learned embedded representations.

Why Similar Words End Up Close

When the corpus repeatedly contains pairs such as

(bank, loan), (bank, interest) and (lender, loan), (lender, interest),

both *bank* and *lender* are trained against the **same context words**.

Their rows in W therefore receive similar gradient updates, and over time:

$$e_{\text{in}}(\text{bank}) \approx e_{\text{in}}(\text{lender}).$$

Distributional consequence

Geometric proximity is not assumed. It **emerges** because similar prediction pressure produces similar parameter updates.

Why Similar Words End Up Close

The pretext task and self-supervised learning

Word2Vec is **not** a language model: nobody uses its predictions. The prediction task is a *pretext* — its sole role is to generate the gradient signal that shapes W . This is the first example in the course of **self-supervised learning**, a logic that reappears in BERT and GPT.

GloVe and the Role of Global Co-occurrence

Why Another Embedding Method?

Word2Vec relies primarily on **local context windows**. This is powerful, but one may ask whether the representation should also exploit more explicitly the **global co-occurrence structure** of the corpus.

GloVe idea

GloVe, or **Global Vectors for Word Representation**, combines the embedding philosophy with global corpus statistics derived from word co-occurrence.

The Co-occurrence Matrix X

Count how often word j appears in the context of word i (within a sliding window of, say, 10 words).

	ice	steam	water	solid	gas
ice	—	12	58	47	8
steam	12	—	63	3	52
water	58	63	—	21	19
solid	47	3	21	—	1
gas	8	52	19	1	—

Notation

X_{ij} : # times j appears near i

$X_i = \sum_k X_{ik}$: total context count

$P_{ij} = X_{ij} / X_i$: probability of j given context i

Think of it as a giant spreadsheet: rows are words, columns are words, and each cell records how often these two words appear near each other in the corpus.

GloVe's Key Insight: Ratios Reveal Meaning

Raw probabilities are noisy; **ratios** cancel out noise and isolate the semantic signal.

Probe word k	$P(k \text{ice})$	$P(k \text{steam})$	$P_{\text{ice},k} / P_{\text{steam},k}$
solid	1.9×10^{-4}	2.2×10^{-5}	8.9 ($\gg 1$)
gas	6.6×10^{-5}	7.8×10^{-4}	0.085 ($\ll 1$)
water	3.0×10^{-3}	2.2×10^{-3}	1.36 (≈ 1)
fashion	1.7×10^{-5}	1.8×10^{-5}	0.96 (≈ 1)

$\gg 1$

k relates more to **ice**

$\ll 1$

k relates more to **steam**

≈ 1

k is neutral to both

From Ratios to a Model — Setup

The **ratio** P_{ik}/P_{jk} carries a clean semantic signal. We want to learn **vectors** that capture it.

Three kinds of vectors

- ▶ $\mathbf{w}_i \in \mathbb{R}^d$ — the **target vector** for word i (the word we are describing).
- ▶ $\mathbf{w}_j \in \mathbb{R}^d$ — the **target vector** for a second word j (used for comparison with i).
- ▶ $\tilde{\mathbf{w}}_k \in \mathbb{R}^d$ — the **context vector** for a probe word k (the word we use to “test” the relationship). The tilde marks it as belonging to a *separate* set of vectors.

Recall: co-occurrence notation

X_{ik} : how many times k appears near i . $X_i = \sum_k X_{ik}$: total context count. $P_{ik} = X_{ik}/X_i$: probability of k given context i .

Why two vector sets ($\mathbf{w}$ and $\tilde{\mathbf{w}}$)? A word plays two roles: sometimes it is the target, sometimes it is part of the surrounding context.

From Ratios to a Model — Derivation

Goal: find F such that $F(\mathbf{w}_i, \mathbf{w}_j, \tilde{\mathbf{w}}_k) = P_{ik}/P_{jk}$.

Step 1 — Use the *difference* $\mathbf{w}_i - \mathbf{w}_j$, not both vectors separately.

The ratio P_{ik}/P_{jk} measures how i and j *differ* w.r.t. the probe k , so F should depend on $\mathbf{w}_i - \mathbf{w}_j$:

$$F(\mathbf{w}_i - \mathbf{w}_j, \tilde{\mathbf{w}}_k) = P_{ik} / P_{jk}$$

Step 2 — Reduce two vectors to a scalar via a dot product.

F must output a *number*, but $(\mathbf{w}_i - \mathbf{w}_j)$ and $\tilde{\mathbf{w}}_k$ are vectors. The dot product is the simplest reduction:

$$F((\mathbf{w}_i - \mathbf{w}_j)^\top \tilde{\mathbf{w}}_k) = P_{ik} / P_{jk}$$

Step 3 — Set $F = \exp$, so that subtraction becomes a ratio.

Since $\exp(a - b) = e^a/e^b$, taking logs on both sides gives:

$$\mathbf{w}_i^\top \tilde{\mathbf{w}}_k = \log P_{ik} = \log X_{ik} - \log X_{ij}$$

Absorbing the Normalisation

The $\log X_i$ term in $\mathbf{w}_i^\top \tilde{\mathbf{w}}_k = \log X_{ik} - \log X_i$ breaks the symmetry between $\mathbf{w}$ and $\tilde{\mathbf{w}}$.

Solution: introduce **bias terms** b_i and $\tilde{b}_j$ that absorb $\log X_i$ (and its symmetric counterpart):

$$\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j = \log X_{ij}$$

This is the **core modelling equation** of GloVe:

- ▶ $\mathbf{w}_i^\top \tilde{\mathbf{w}}_j$ captures the interaction between word i and context word j .
- ▶ b_i absorbs the “popularity” of word i (how often it appears overall).
- ▶ $\tilde{b}_j$ absorbs the popularity of context word j .

Intuition

The biases act like **intercepts in a regression**: they soak up the overall frequency effects so that the dot product can focus purely on the *relationship* between the two words.

The GloVe Objective Function

Loss function

$$J = \sum_{i,j=1}^V f(X_{ij}) (\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

$$\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j$$

Model's prediction of $\log X_{ij}$

$$\log X_{ij}$$

Observed target (log
co-occurrence count)

$$f(X_{ij})$$

Weighting function (controls
each pair's influence)

Read it as: “find vectors whose dot products match the log co-occurrence counts, weighting frequent pairs more.”

The Weighting Function $f(x)$

$$f(x) = \begin{cases} (x / x_{\max})^\alpha & \text{if } x < x_{\max} \\ 1 & \text{if } x \geq x_{\max} \end{cases}$$

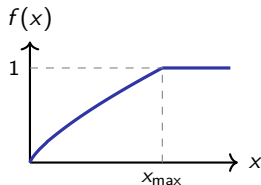
Why do we need this?

Without weighting, ultra-common pairs like (“the”, “of”) would dominate the loss and drown out the signal from rarer, more meaningful pairs.

The function f acts as a **volume knob**: it turns down the influence of the most frequent pairs and gives rare pairs a fair chance.

Design Properties

- ▶ $f(0) = 0$ — skip zero-count pairs
- ▶ Non-decreasing — frequent pairs still matter
- ▶ Saturates at $x_{\max}$ — prevents “the” from dominating
- ▶ Typical: $\alpha = 3/4$, $x_{\max} = 100$



Training GloVe in Practice

- 1 **Build the co-occurrence matrix** — Scan the entire corpus with a sliding window. Store all non-zero X_{ij} entries (sparse matrix).
- 2 **Initialise vectors randomly** — Two sets of vectors ($\mathbf{w}$ and $\tilde{\mathbf{w}}$) plus biases $b_i, \tilde{b}_j$. Dimension d is typically 50, 100, 200, or 300.
- 3 **Minimise J with AdaGrad** — Sample non-zero entries, compute gradients, update all parameters. AdaGrad adapts the learning rate per parameter — important because word frequencies span many orders of magnitude.
- 4 **Combine the two vector sets** — Final embedding = $\mathbf{w}_i + \tilde{\mathbf{w}}_i$. Averaging the two sets improves quality because the model treats them symmetrically.

Unlike Word2Vec, GloVe does **not** need multiple passes over raw text — it only iterates over the (much smaller) set of **non-zero matrix entries**.

Key Hyperparameters

Parameter	Typical	Effect
d (dimension)	100–300	Larger $\rightarrow$ richer, but more data needed
Window size	5–15	Larger $\rightarrow$ captures more topical similarity
$x_{\max}$	100	Cap on weighting function saturation
α	0.75	Sub-linear scaling of co-occurrence counts
Iterations	50–100	More iterations $\rightarrow$ better convergence
Learning rate	0.05	AdaGrad handles per-parameter adaptation

Pre-trained vectors

The Stanford NLP Group provides pre-trained GloVe vectors on:

- ▶ Wikipedia 2014 + Gigaword 5 (6B tokens, 400K vocab)
- ▶ Common Crawl (42B and 840B tokens)

Available at: <https://nlp.stanford.edu/projects/glove/>

Word Analogies: Linear Structure

GloVe vectors encode semantic relationships as **vector arithmetic**.

Gender

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \text{queen}$$

Geography

$$\vec{\text{Paris}} - \vec{\text{France}} + \vec{\text{Italy}} \approx \text{Rome}$$

Tense

$$\vec{\text{walking}} - \vec{\text{walked}} + \vec{\text{swam}} \approx \text{swimming}$$

This works because GloVe's log-bilinear structure naturally encodes differences as directions in vector space.

Benchmark Performance

Word Analogy Task (semantic + syntactic)

Model	Dim	Corpus size	Accuracy (%)
Word2Vec Skip-gram	300	1B	61.0
GloVe	300	6B	71.7
GloVe	300	42B	75.0

Source: Pennington et al. (2014), Table 2

Word Similarity Tasks (Spearman correlation)

GloVe achieves competitive or state-of-the-art results on WordSim-353, RG-65, MC-30, and other standard benchmarks, with performance scaling consistently with both corpus size and vector dimensionality.

More data and more dimensions help, but with diminishing returns beyond $d \approx 300$.

	GloVe	Word2Vec (Skip-gram)
Approach	Count-based + regression	Prediction-based (neural net)
Training data	Co-occurrence matrix (pre-computed)	Raw text (online, window-by-window)
Statistics	Global (entire corpus at once)	Local (one window at a time)
Objective	Weighted least squares on log counts	Softmax / Negative sampling
Parallelism	Easy (matrix entries are independent)	Harder (sequential text)
Performance	Comparable; often wins on analogy	Comparable; often wins on similarity

A deeper connection

Levy & Goldberg (2014) showed that Skip-gram with negative sampling **implicitly factorises a shifted PMI matrix** — the two approaches are more similar than they appear.

Applications in NLP and Finance

Semantic Search

Find documents by **meaning**, not just keyword matching. Queries and documents live in the same vector space.

Named Entity Recognition

Embeddings help detect company names, tickers, and financial entities in unstructured text (news, filings, reports).

Sentiment Analysis

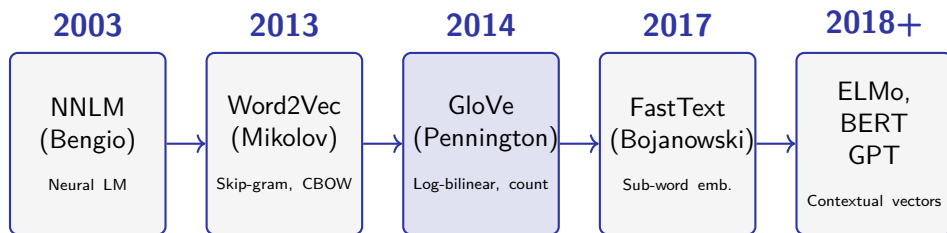
Pre-trained embeddings provide a strong initialisation for financial sentiment classifiers, even with limited labelled data.

Risk Factor Analysis

Cluster SEC filings by embedding similarity to discover emerging risk themes across industries and time periods.

In practice, GloVe embeddings are often used as the **initialisation layer** for downstream deep learning models (LSTMs, Transformers).

GloVe in the Embedding Landscape



Key takeaway

GloVe produces **static** embeddings (one vector per word regardless of context). Modern Transformers generate **contextual** embeddings, but GloVe remains valuable for its simplicity, interpretability, and efficiency.

Glove: Key Takeaways

- 1 Word embeddings turn words into **dense vectors** where geometry encodes meaning.
- 2 GloVe's key insight: **ratios of co-occurrence probabilities** distinguish relevant from irrelevant context words.
- 3 The objective is a **weighted least-squares regression** on log co-occurrence counts — elegant and easy to optimise.
- 4 The weighting function f prevents common words from dominating the loss.
- 5 GloVe vectors capture **linear relationships** (analogies) and transfer well to downstream tasks.
- 6 Though superseded by contextual models (BERT, GPT), GloVe remains a **clean, efficient, and interpretable** baseline.

Pre-Trained Embedding Models

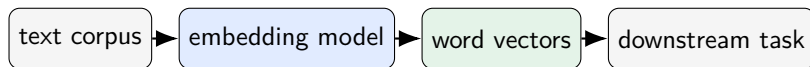
Pre-trained Embedding Models

- ▶ Training embeddings on a very large corpus can require substantial time and resources.
- ▶ For this reason, practitioners often use **pre-trained** Word2Vec or GloVe vectors.
- ▶ The idea is simple: representation learning is expensive, so we reuse embeddings learned from large generic corpora.

Practical consequence

Pre-trained embeddings made it possible to bring dense semantic information into downstream models even when the task-specific dataset was relatively small.

Embeddings in the Modeling Pipeline



- ▶ sentiment analysis,
- ▶ document classification,
- ▶ similarity,
- ▶ search,
- ▶ sequence labeling,
- ▶ many other downstream tasks.

Limits of Static Embeddings

Domain Mismatch: Why General Embeddings May Mislead in Finance

Pre-trained Word2Vec and GloVe vectors are typically trained on Wikipedia or Google News.

General corpus

- ▶ *bond* → near *James Bond, movie, spy*
- ▶ *call* → near *phone, dial, ring*
- ▶ *alpha* → near *Greek, beta, alphabet*

Financial corpus

- ▶ *bond* → near *yield, coupon, maturity*
- ▶ *call* → near *option, strike, put*
- ▶ *alpha* → near *excess return, benchmark*

Implication for quantitative finance

Using general-purpose embeddings on financial text can produce semantically misleading similarities. This motivates either **domain-specific training** (embeddings on 10-K filings, earnings calls, financial news) or **fine-tuning** — a theme we will revisit with contextual models like FinBERT in Lesson 3.

The Main Limitation of Static Embeddings

One word, one vector

In Word2Vec and GloVe, each word type is assigned a single vector, no matter where the word appears.

- ▶ This is elegant and useful.
- ▶ But natural language is highly context-dependent.
- ▶ Many words are polysemous: they have multiple meanings.

Why Context Eventually Becomes Central

Consider the word *bank*:

- ▶ in “*the bank approved the loan*”, it refers to a financial institution;
- ▶ in “*the fisherman sat on the bank*”, it refers to a river bank.

Problem

A static embedding gives both occurrences the same vector.

Implication

If representation is to follow meaning more closely, the model must learn to modify a word's embedding according to its context.

Another Example: *current*

General representation

A static embedding may place *current* close to several semantic neighborhoods at once:

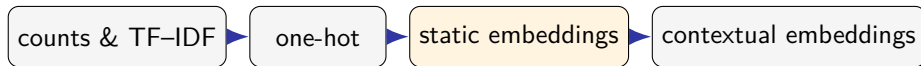
- ▶ *news*
- ▶ *present*
- ▶ *electricity*

Context-sensitive need

In a sentence about circuits, the relevant relation should privilege the link between *current* and *electricity*, not the link between *current* and *news*.

This is one of the conceptual pressures that eventually leads toward **contextual embeddings** and later toward **self-attention**.

A Historical Summary of the Transition



Interpretation

Today's lesson has positioned us at the threshold of the transformer era: we now understand why distributed representations are necessary, why neural networks matter, and why static vectors are still not enough.

Conclusion and Bridge to Lesson 2b

Conclusion: What We Have Built Today

- ▶ We introduced the idea of representation learning through neural models.
- ▶ We studied the logic of dense word embeddings, Word2Vec, and GloVe.
- ▶ We understood why static embeddings are powerful yet limited.

Compact summary

Lesson 1 asked how text can be represented at all. Lesson 2a asked how a model can **learn** such a representation. The next lesson will another fundamental property of language: sequentiality.

Main Takeaways

- ① Neural networks matter in NLP not only as classifiers, but as **representation learners**.
- ② Word embeddings transform symbolic words into dense vectors whose geometry reflects contextual similarity.
- ③ Word2Vec and GloVe are historically decisive because they made semantic geometry operational.
- ④ Their main weakness is that they are **static**: one word still receives one vector.

What We Have Built — and What Is Still Missing

What Lesson 2a established

- ▶ Differentiable neurons enable gradient-based learning.
- ▶ Hidden layers learn **intermediate representations**, not just classifiers.
- ▶ Word2Vec and GloVe show that **semantic geometry emerges** from predictive pressure on large corpora.

What remains open

- ▶ Static embeddings assign **one vector per word**, regardless of context.
- ▶ They treat the vocabulary as a set of independent symbols — **no notion of order or sequence**.
- ▶ A model that reads language must process tokens **one after another**, accumulating meaning as it goes.

The question for Lesson 2b

How can a neural model process a sentence of **arbitrary length**, retain information from earlier tokens, and eventually produce an output — possibly of a *different* length? This requires a **fundamentally new architectural idea: recurrence**.

References

-  Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *Proc. EMNLP*, 1532–1543.
-  Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. *NeurIPS*.
-  Levy, O. & Goldberg, Y. (2014). Neural Word Embedding as Implicit Matrix Factorization. *NeurIPS*.
-  Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching Word Vectors with Subword Information. *TACL*, 5, 135–146.
-  Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers. *NAACL-HLT*.